AUSTCAIC — AI & ML WING

Class 3: Machine Learning Fundamentals

Session Date: 1st August • Time: 9:00 PM

OVERVIEW

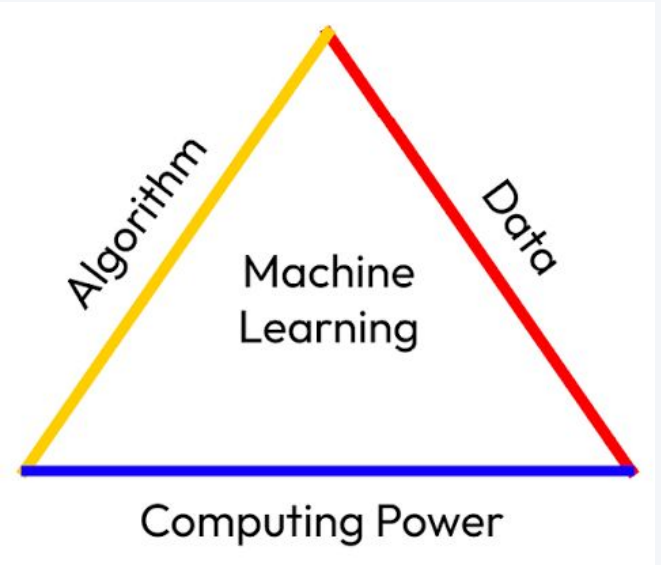
What We'll Cover Today

1. Supervised vs Unsupervised
2. Regression vs Classification
3. The Machine Learning Pipeline
4. Train/Test Split
5. Common ML Algorithms
6. Regression Metrics (MAE, RMSE)
7. Classification Metrics & Confusion Matrix
8. Overfitting vs Underfitting

RECAP

Quick Recap: Class 2

- We covered EDA: inspecting data, features vs target, value counts, and visualization.
- We covered preprocessing: missing values, duplicates, feature scaling, and encoding.
- We learned that clean, well-prepared data is what feeds into a machine learning model.
- **Today: we finally build and evaluate the models themselves.**



Supervised vs Unsupervised Learning

Supervised Learning

- Learns from labeled data — features and a known target/answer
- The model learns to map inputs to known outputs
- Used when you have historical outcomes to learn from
- Examples: predicting house prices, loan approval, spam detection

Unsupervised Learning

- Learns from unlabeled data — no known target/answer
- The model finds hidden patterns or groupings on its own
- Used when you want to discover structure, not predict a known label
- Examples: customer segmentation, anomaly detection

Regression vs Classification

Both are types of supervised learning — the difference is what kind of value you're predicting.

Regression

- Predicts a continuous numeric value
- Output can be any number in a range
- Example: predicting a house's price ($) or a student's exam score

Classification

- Predicts a discrete category or class
- Output is one of a fixed set of labels
- Example: predicting loan approval (Yes/No) or email spam (Spam/Not Spam)

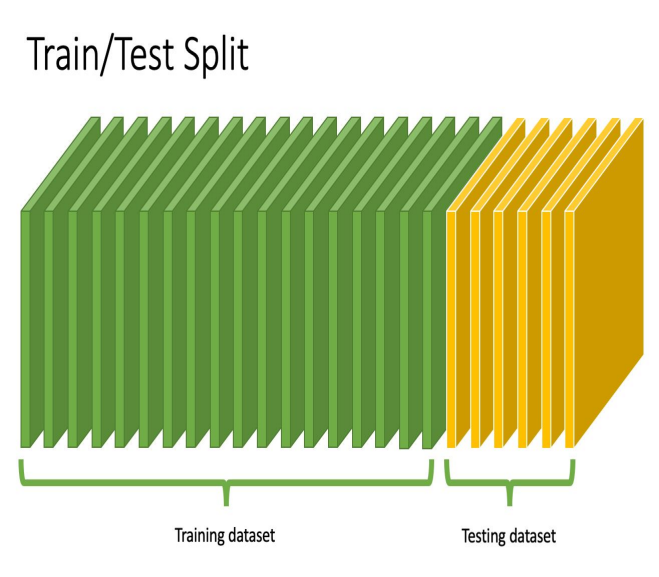
The Machine Learning Pipeline



- Split: divide data into a training set and a testing set.
- Train: the model learns patterns from the training set only.
- Predict: the trained model makes predictions on new/unseen data.
- Evaluate: compare predictions to actual answers to measure performance.

Why We Split: Train/Test Split

- We never test a model on the same data it was trained on — that would be like grading a student on questions they already saw the answers to.
- Training set: the larger portion (commonly 70–80%) used to teach the model.
- Testing set: the remaining portion (commonly 20–30%) used to check how well it generalizes to new data.
- The split should be random, so both sets represent the overall data fairly.



Train/Test Split in Scikit-Learn

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print(X_train.shape, X_test.shape)
```

Training & Prediction

Training (.fit())

- The model looks at X_train and y_train together
- It adjusts its internal parameters to capture the relationship between features and target
- This is the 'learning' step — the model has not seen the test set yet

Prediction (.predict())

- The trained model is given X_test only (no answers)
- It outputs predicted values: y_pred
- We then compare y_pred against the real y_test to judge performance

Evaluation: How Do We Know It's Good?

- Evaluation compares the model's predictions (y_pred) to the actual answers (y_test).
- Different problem types need different metrics — a regression model and a classification model are judged differently.
- We'll cover the exact metrics in detail later in this class: MAE, MSE, RMSE for regression, and Accuracy, Precision, Recall, Confusion Matrix for classification.
- For now, just remember: no model is complete without evaluating it on unseen data.

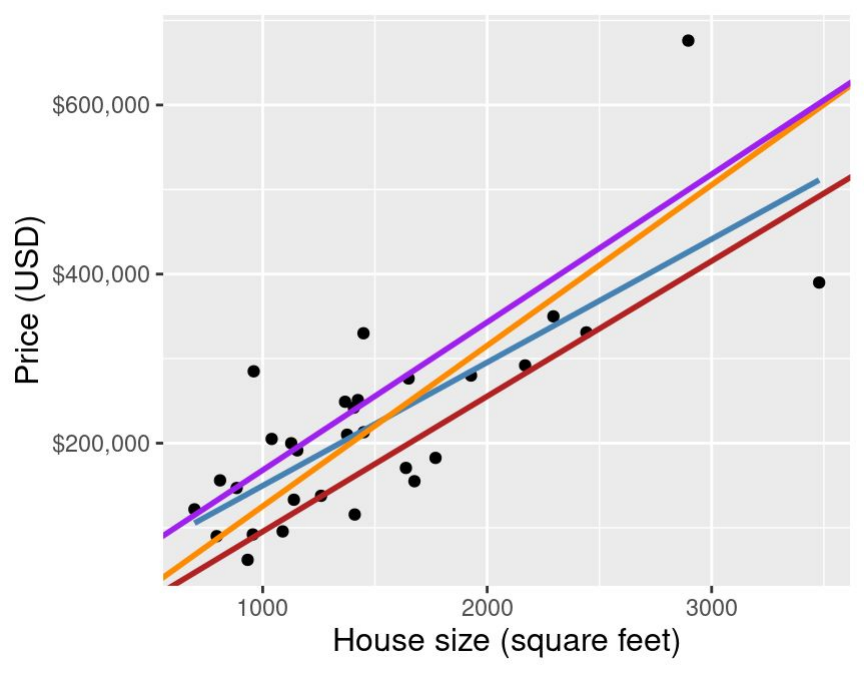
03 · ALGORITHMS

Common Machine Learning Algorithms

Linear Regression	Predicts a continuous number using a straight-line relationship
Logistic Regression	Predicts a category (despite the name) using a probability curve
Decision Tree	Splits data into branches using a series of yes/no questions
K-Means Clustering	An Example for Unsupervised Learning
K-Nearest Neighbors	Classifies a point based on its closest neighbors in the data

Linear Regression

- Used for regression problems — predicting a continuous number.
- Finds the best-fitting straight line through the data points.
- Formula: y = mx + b (one feature) or y = w1x1 + w2x2 + ... + b (many features).
- Example: predicting a house's price based on its size.



Linear Regression in Scikit-Learn

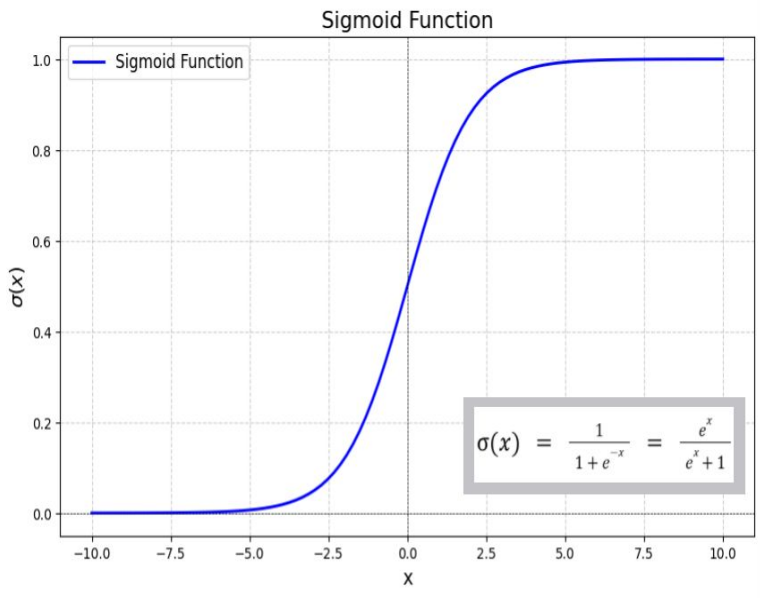
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(y_pred[:5])
```

Logistic Regression

- Despite the name, it's used for classification, not regression.
- Outputs a probability (0 to 1) that an input belongs to a given class.
- Uses an S-shaped curve (sigmoid function) instead of a straight line.
- Example: predicting whether a loan will be approved (Yes/No).



Logistic Regression in Scikit-Learn

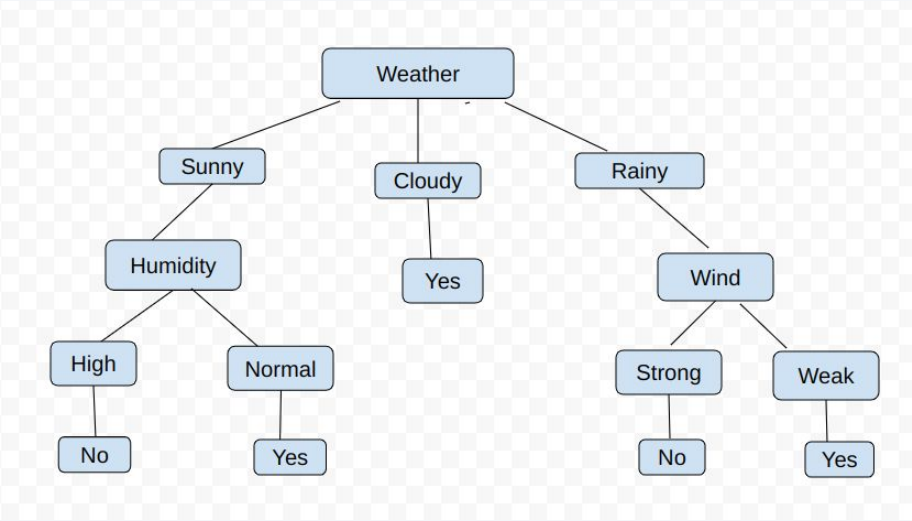
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(y_pred[:5])
```

Decision Tree

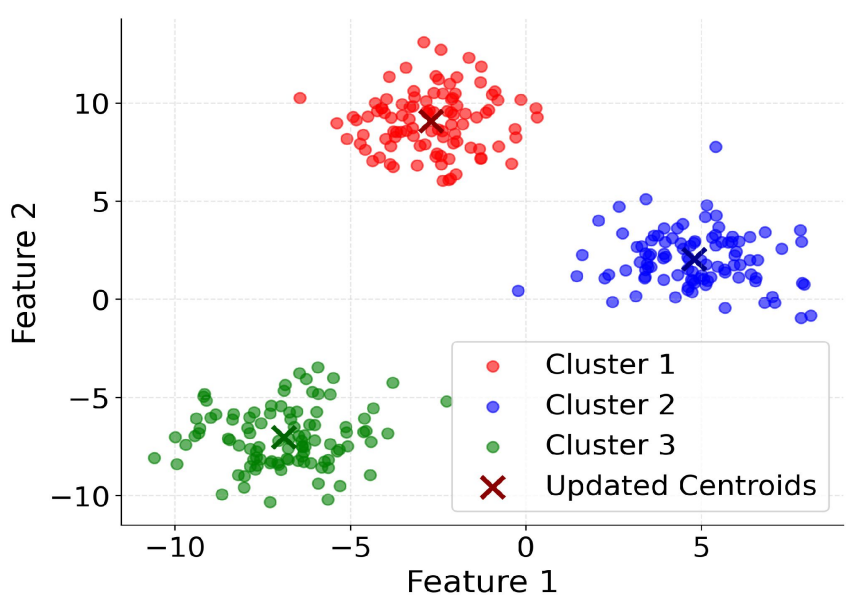
- Splits data step by step using simple yes/no questions on features.
- Can be used for both regression and classification.
- Easy to visualize and interpret — like a flowchart of decisions.
- Downside: can overfit easily if allowed to grow too deep.



Day	Weather	Humidity	Wind	Play Football?
Day 1	Sunny	High	Weak	**No**
Day 2	Sunny	Normal	Strong	**No**
Day 3	Cloudy	Yes	Weak	**Yes**
Day 4	Rainy	Yes	Strong	**Yes**
Day 5	Rainy	Normal	Weak	**Yes**
Day 6	Sunny	High	Strong	**No**

K-Means Clustering

- An unsupervised algorithm — it groups unlabeled data based on similarity, with no target to predict.
- Picks 'k' cluster centers (centroids), then assigns each point to its nearest centroid.
- Centroids are updated repeatedly until cluster assignments stop changing.
- Example: grouping customers into segments by purchasing behavior, with no predefined labels.



Algorithm Cheat Sheet

Algorithm	Type	Best For
Linear Regression	Regression	Simple numeric predictions with linear relationships
Logistic Regression	Classification	Binary outcomes (Yes/No, Spam/Not Spam)
Decision Tree	Both	Interpretable rules, quick prototyping
Random Forest	Both	Higher accuracy, resistant to overfitting
K-Nearest Neighbors	Both	Simple pattern-based classification, scaled data

Evaluating Regression Models

MAE

- Mean Absolute Error
- Average of the absolute differences between predicted and actual values
- Easy to interpret: same unit as the target

MSE

- Mean Squared Error
- Average of the squared differences between predicted and actual values
- Penalizes large mistakes more than MAE, since errors are squared

RMSE

- Root Mean Squared Error
- Like MAE, but squares errors first — penalizes large mistakes more
- Sensitive to outliers

Regression Metrics in Scikit-Learn

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error
)

mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False)

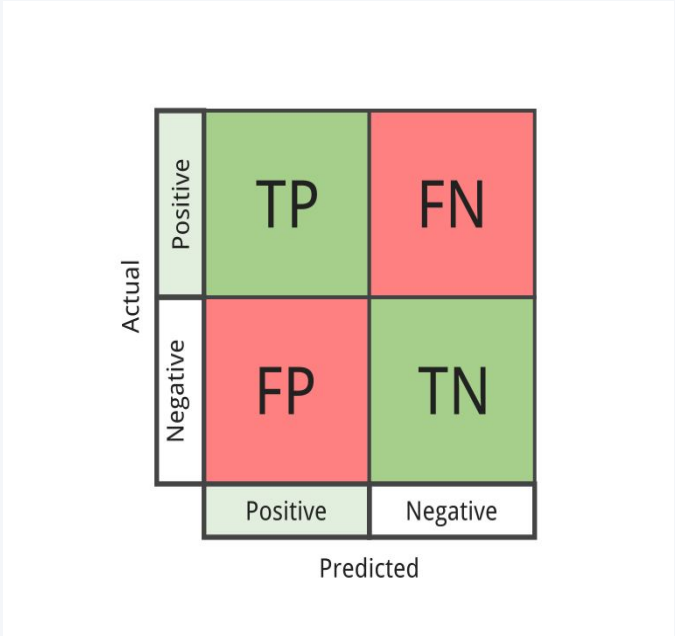
print(f'MAE: {mae}, RMSE: {rmse}')
```

Evaluating Classification Models

- **Accuracy:**
- Percentage of predictions that were correct overall.
- **Precision:**
- Of everything predicted positive, how many were actually positive?
- **Recall:**
- Of everything that was actually positive, how many did we correctly catch?
- Accuracy alone can be misleading on imbalanced data — precision and recall tell a fuller story.

The Confusion Matrix

- A table showing correct vs incorrect predictions, broken down by class.
- **True Positive (TP):**
- Predicted positive, actually positive.
- **False Positive (FP):**
- Predicted positive, actually negative.
- **False Negative (FN):**
- Predicted negative, actually positive.
- **True Negative (TN):**
- Predicted negative, actually negative.



Classification Metrics in Scikit-Learn

```
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, confusion_matrix
)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(acc, prec, rec)
print(cm)
```

Overfitting vs Underfitting

Overfitting

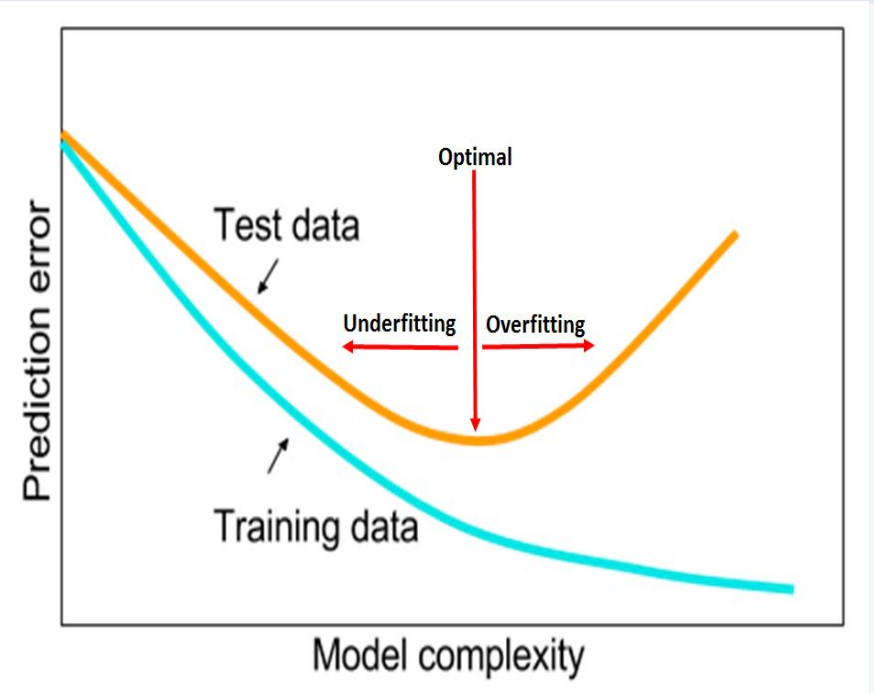
- Model learns the training data too well, including its noise
- Performs great on training data, poorly on test data
- Sign: high training accuracy, low test accuracy
- Fix: simplify the model, get more data, or use regularization

Underfitting

- Model is too simple to capture the underlying pattern
- Performs poorly on both training and test data
- Sign: low accuracy everywhere
- Fix: use a more complex model, or add more relevant features

Finding the Right Balance

- Goal: a model that generalizes well — not just memorizes, not just guesses.
- This balance is often visualized as a curve of model complexity vs error.
- Too simple → underfitting. Too complex → overfitting. The sweet spot is in between.
- Comparing training vs test performance is the simplest way to detect either problem.



SUMMARY

Key Takeaways

- Supervised learning uses labeled data; regression predicts numbers, classification predicts categories.
- Every ML workflow follows the same pipeline: split, train, predict, evaluate.
- Different algorithms suit different problems — start simple (Linear/Logistic Regression) before reaching for complexity (Random Forest).
- Use the right metric for the job: MAE/RMSE/MSE for regression, Accuracy/Precision/Recall for classification.
- Always check for overfitting vs underfitting by comparing training and test performance.